Parcial Algoritmos y Programación II - 7541 Curso Calvo (1º instancia) 25 de noviembre de 2016

Padrón: **Apellido y Nombre:**

Punteros: APROBADO – DESAPROBADO **TDA:** APROBADO – DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main() {

    int *A, *C, *F;
    int **B;
    char *D, *E;
    int H;

    H = 65 + ULTIMO_DIGITO_PADRON;
    A = new int;
    (*A) = H;
    F = &H;
    cout << (*F) << (*A) << endl;
    ;
    B = &F;
    (**B) = 65;
    cout << (*F) << (*A) << (**B) << endl;

    F = new int[3];
    (**B) = 70;

    for (int i = 0; i < 3; i++) {
        F[i] = (*A) + i;
    }
    cout << (*F) << (**B) << endl;

    D = (char*) A;
    E = (char*) (F + 1);
    C = (*B);
    C[1] = H;

    cout << (*D) << (*E) << (*C) << endl;
    (*D) = 'B';
    (*E) = 'B';

    for (int i = 0; i < 3; i++) {
        F[i] = 65;
    }
    C = C + 2;
    cout << (*D) << (*E) << (*C) << endl;

    //liberar memoria
    return 0;
}
```

2. Implementar para la clase Cola<T> las primitivas, indicando pre y post condiciones:

```
void acolar(T elemento);
T desacolar();
```

3. Implementar el método `'buscarSolucionEquivalente'` de la clase `'IngenieroQuimico'` a partir de las siguientes especificaciones:

```
class IngenieroQuimico {

  public:

    /* post: busca en 'solucionesDisponibles' una Solución que tenga los mismos Compuestos que 'solucionRequerida',
     *       con cantidades iguales o superiores.
     */
    Solucion* buscarSolucionEquivalente(Solucion* solucionRequerida, Lista<Solucion*>* solucionesDisponibles)
};
```

```
class Solucion {

  public:

   /* post: crea la solución con el código especificado, sin
    *       compuestos asociados
    */
    Solucion(string codigo);

    string obtenerCodigo();

    /* post: devuelve los Compuestos requeridos para preparar
     *       la Solución
     */
    Lista<Compuesto*>* obtenerCompuestos()

};
enum Unidad {

  KILO; LITRO;
};
```

```
class Compuesto {
  public:

    /* post: crea el Compuesto, identificado por 'nombre',
     *       con cantidad 0 de la Unidad 'unidadDeMedida'
     */
    Compuesto(string nombre, Unidad unidadDeMedida);

    /* post: devuelve el nombre que identifica el Compuesto
     */
    string obtenerNombre();

    /* post: devuelve la Unidad en la que se mide la cantidad.
     */
    Unidad obtenerUnidad();

    float obtenerCantidad();

    void cambiarCantidad(float cantidad);
};
```

4. Diseñar la especificación e implementar el TDA Colecta. Debe proveer operaciones para:

- crear la Colecta recibiendo como parámetro el monto [$] considerado como objetivo de la misma y los montos [$] de la mínima [$] y máxima donación individual aceptada.
- calcularRecaudación: devuelve el monto total [$] recaudado.
- donar: recibe el monto [$] a donar y lo agrega a la recaudación.
- contarDonaciones: devuelve la cantidad de donaciones recibidas.
- calcularDonacionMaxima: devuelve el monto [$] de la máxima donación recibida.
- calcularRecaudacionFaltante: devuelve el monto [$] que falta recaudar para cumplir con el objetivo de la Colecta, o cero [$] en caso de haberse superado.

Los alumnos que tienen aprobado el parcialito de punteros/TDA no deben realizar el ejercicio 1/4.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas

0
6565
656565
6565
AA65
BA65
1
6666
656665
6666
BA66
BA65
2
6767
656765
6767
CA67
BA65
3
6868
656865
6868
DA68
BA65
4
6969
656965
6969
EA69
BA65
5
7070
657065
7070
FA70
BA65
6
7171
657165
7171
GA71
BA65
7
7272
657265
7272
HA72
BA65
8
7373
657365
7373
IA73
BA65
9
7474
657465
7474
JA74
BA65

delete A;
delete[] F;

Los alumnos que tienen aprobado el parcialito de punteros/TDA no deben realizar el ejercicio 1/4.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas